

RePlot

Land Readjustment & Replotting Software

Comprehensive Undo/Redo System

Architecture, Commands, and Implementation Guide for the Map Canvas

Version 1.0 | March 2026

Chapter 1: What You Have Now and What Is Missing

1.1 Your Current Undo/Redo Architecture

Your existing system has three parts: an ICommand interface, an UndoRedoManager that maintains two stacks, and three concrete command classes. Here is exactly what exists:

Component	What It Does	Status
ICommand	Interface with Execute, Undo, Redo, CanMergeWith, MergeWith, Description	Complete and correct
UndoRedoManager	Two stacks (undo/redo), merge support, max undo levels, push/pop logic	Complete and correct
AddShapeCommand	Adds one shape via ShapeManager.AddShape(). Undo removes it.	Working
BulkAddShapesCommand	Adds many shapes via BulkAddShapes(). Undo bulk-removes.	Working
DeleteShapesCommand	Removes shapes. Undo re-adds them.	Working
ClearAllCommand	Clears all shapes. Undo restores from snapshot.	Working

1.2 What Is NOT Undoable Right Now

Every other canvas operation bypasses the undo system entirely. When a user performs these actions, pressing Ctrl+Z does nothing:

Operation	Current Behavior	User Expectation
Zoom In/Out	Directly calls _engine.ZoomAtPoint(). No command.	Ctrl+Z returns to previous zoom level
Pan	Directly updates _viewOffset. No command.	Ctrl+Z returns to previous view position
Zoom to Extents	Directly calls ZoomToExtents(). No command.	Ctrl+Z returns to previous view
Move Shape (drag)	Not implemented yet	Ctrl+Z returns shape to original position
Modify Shape Property	Direct property change (color, layer, etc.)	Ctrl+Z restores original property value
DXF Import	Uses BulkAddShapesCommand (correct)	Already works

Operation	Current Behavior	User Expectation
Polyline Drawing	Uses AddShapeCommand when finished (correct)	Already works
Parcel Split (future)	Not implemented yet	Ctrl+Z restores original parcel
Parcel Merge (future)	Not implemented yet	Ctrl+Z separates merged parcels

The Core Problem

Your ICommand interface and UndoRedoManager are well-designed. The problem is that only 4 out of 12+ canvas operations actually create commands. Everything else bypasses the undo system. This guide shows you exactly which commands to create and how to wire them.

Chapter 2: Design Decisions for Version 1.0

2.1 Should Zoom and Pan Be Undoable?

This is the most important design decision and different professional applications handle it differently:

Application	Zoom Undoable?	Pan Undoable?	Approach
AutoCAD	No	No	Separate View stack
QGIS	No	No	View history (back/forward)
Photoshop	No	No	View is not a document change
ETABS	No	No	View bookmarks
Figma	No	No	Zoom to selection shortcut

Industry Consensus

No professional application puts zoom/pan on the main undo stack. The reason: users press Ctrl+Z to undo work (adding shapes, moving parcels, deleting boundaries). If zoom clutters the undo stack, a user who zooms 10 times then draws a line needs to press Ctrl+Z 11 times to undo the line. This is extremely frustrating.

The RePlot Decision: Separate View History Stack

We will implement a separate ViewHistoryManager that tracks zoom/pan states independently. The user gets a "Previous View" / "Next View" feature (like a browser back/forward button) without polluting the undo stack. This is exactly what AutoCAD does.

2.2 What Goes on the Undo Stack (Version 1.0)

Only operations that modify the drawing document (shapes, properties, geometry) go on the undo stack. View changes (zoom, pan, theme) go on the separate view stack.

Undo Stack (Ctrl+Z)	View Stack (Alt+Left/Right)	Neither (Transient)
Add shape (draw)	Zoom in/out (wheel)	Mouse hover position
Delete shapes	Pan (drag)	Snap glyph display
Clear all	Zoom to extents	Preview shape (rubber band)
Import DXF (bulk add)	Zoom to selection	Grid visibility toggle
Move shape (drag)	Set specific zoom level	Theme change (dark/light)
Modify property (color, layer)		Debug overlay toggle
Parcel split (future)		Toolbar state

Undo Stack (Ctrl+Z)	View Stack (Alt+Left/Right)	Neither (Transient)
Parcel merge (future)		Cursor change

2.3 Command Merge Strategy

Your ICommand already has CanMergeWith() and MergeWith(). Here is when to use them:

- **Move Shape:** While the user drags a shape, every MouseMove fires a move delta. These MUST merge into a single command. Otherwise, dragging a shape 200 pixels creates 200 undo steps.
- **Consecutive Zoom:** Scrolling the mouse wheel 5 times should NOT create 5 view history entries. Merge zooms within a 500ms window into one entry.
- **All other commands:** Do NOT merge. Each add, delete, split, or property change is its own undo step.

Chapter 3: View History Manager (Zoom/Pan Undo)

3.1 The ViewState Snapshot

A view state captures exactly two things: where you are looking (offset) and how zoomed in you are (scale). This is all DrawingEngine needs to restore any view.

File: MapCanvas/Core/ViewState.cs

```
public readonly struct ViewState
{
    public double ZoomScale { get; }
    public double OffsetX { get; }
    public double OffsetY { get; }
    public DateTime Timestamp { get; }

    public ViewState(double zoom, double offsetX, double offsetY)
    {
        ZoomScale = zoom;
        OffsetX = offsetX;
        OffsetY = offsetY;
        Timestamp = DateTime.UtcNow;
    }

    // Two states are "same" if zoom and offset are nearly identical
    public bool IsSameAs(ViewState other, double tolerance = 0.001)
    {
        return Math.Abs(ZoomScale - other.ZoomScale) < tolerance
            && Math.Abs(OffsetX - other.OffsetX) < tolerance
            && Math.Abs(OffsetY - other.OffsetY) < tolerance;
    }
}
```

Why a struct?

ViewState is a small, immutable value type (24 bytes). Allocating it on the stack avoids GC pressure. You will create hundreds of these during a session and never modify them.

3.2 ViewHistoryManager

File: MapCanvas/Core/ViewHistoryManager.cs

```
public class ViewHistoryManager
{
    private readonly List<ViewState> _history = new();
    private int _currentIndex = -1;
    private readonly int _maxEntries;
    private readonly TimeSpan _mergeWindow;
```

```

public ViewHistoryManager(int maxEntries = 50,
    int mergeWindowMs = 500)
{
    _maxEntries = maxEntries;
    _mergeWindow = TimeSpan.FromMilliseconds(mergeWindowMs);
}

public bool CanGoBack => _currentIndex > 0;
public bool CanGoForward => _currentIndex < _history.Count - 1;

/// <summary>
/// Record the current view. Called AFTER any zoom/pan operation.
/// Merges with previous entry if within the merge window
/// (prevents mouse wheel scroll from creating 20 entries).
/// </summary>
public void RecordView(ViewState state)
{
    // Don't record if identical to current
    if (_currentIndex >= 0
        && _history[_currentIndex].IsSameAs(state))
        return;

    // Merge with previous if within time window
    if (_currentIndex >= 0)
    {
        var prev = _history[_currentIndex];
        if (DateTime.UtcNow - prev.Timestamp < _mergeWindow)
        {
            // Replace the previous entry instead of adding
            _history[_currentIndex] = state;
            return;
        }
    }

    // Trim forward history (like browser: navigate
    // back then do new action removes forward history)
    if (_currentIndex < _history.Count - 1)
        _history.RemoveRange(_currentIndex + 1,
            _history.Count - _currentIndex - 1);

    _history.Add(state);
    _currentIndex = _history.Count - 1;

    // Enforce max entries
    if (_history.Count > _maxEntries)
    {
        _history.RemoveAt(0);
        _currentIndex--;
    }
}

```

```

    /// <summary>
    /// Navigate to previous view. Returns the ViewState to restore.
    /// </summary>
    public ViewState? GoBack()
    {
        if (!CanGoBack) return null;
        _currentIndex--;
        return _history[_currentIndex];
    }

    /// <summary>
    /// Navigate to next view. Returns the ViewState to restore.
    /// </summary>
    public ViewState? GoForward()
    {
        if (!CanGoForward) return null;
        _currentIndex++;
        return _history[_currentIndex];
    }

    public void Clear() { _history.Clear(); _currentIndex = -1; }
}

```

3.3 What Changes in DrawingEngine

DrawingEngine needs one new method to restore a view state, and it needs to expose its internal state for snapshotting:

Add to DrawingEngine.cs:

```

// New: Get current view as a snapshot
public ViewState GetViewState()
{
    return new ViewState(_zoomScale, _viewOffset.X, _viewOffset.Y);
}

// New: Restore a previously saved view
public void RestoreViewState(ViewState state)
{
    _zoomScale = state.ZoomScale;
    _viewOffset.X = state.OffsetX;
    _viewOffset.Y = state.OffsetY;
}

```

3.4 What Changes in DrawingCanvasControl

Add a ViewHistoryManager field and record view after every zoom/pan:

Add field:

```
| private ViewHistoryManager _viewHistory = new();
```

Modify PanelCanvas_MouseWheel:**BEFORE:**

```
| private void PanelCanvas_MouseWheel(object sender, MouseEventArgs e)
| {
|     float zoomFactor = e.Delta > 0
|         ? (float)DrawingEngine.ZOOM_STEP
|         : (float)(1.0 / DrawingEngine.ZOOM_STEP);
|     _engine.ZoomAtPoint(e.Location, zoomFactor);
|     _deferredRenderer.RenderNow();
|     panelCanvas.Invalidate();
| }
```

AFTER (one line added):

```
| private void PanelCanvas_MouseWheel(object sender, MouseEventArgs e)
| {
|     float zoomFactor = e.Delta > 0
|         ? (float)DrawingEngine.ZOOM_STEP
|         : (float)(1.0 / DrawingEngine.ZOOM_STEP);
|     _engine.ZoomAtPoint(e.Location, zoomFactor);
|     _viewHistory.RecordView(_engine.GetViewState()); // NEW
|     _deferredRenderer.RenderNow();
|     panelCanvas.Invalidate();
| }
```

Modify PanelCanvas_MouseUp (end of pan):

After the existing EndPan() call, add:

```
| _viewHistory.RecordView(_engine.GetViewState()); // NEW
```

Modify btnZoomExtents_Click:

After ZoomToExtents(), add:

```
| _viewHistory.RecordView(_engine.GetViewState()); // NEW
```

Add keyboard shortcuts for view navigation:

```
| // In ProcessCmdKey:
| if (keyData == (Keys.Alt | Keys.Left))
| {
|     var prev = _viewHistory.GoBack();
|     if (prev.HasValue)
```

```
        {
            _engine.RestoreViewState(prev.Value);
            _deferredRenderer.RenderNow();
            panelCanvas.Invalidate();
        }
        return true;
    }
    if (keyData == (Keys.Alt | Keys.Right))
    {
        var next = _viewHistory.GoForward();
        if (next.HasValue)
        {
            _engine.RestoreViewState(next.Value);
            _deferredRenderer.RenderNow();
            panelCanvas.Invalidate();
        }
        return true;
    }
}
```

Chapter 4: New Command Classes to Implement

Each command follows the same pattern as your existing `AddShapeCommand`: it captures the minimum state needed to execute and reverse the operation. Here are the commands you need for Version 1.0, in priority order.

4.1 MoveShapeCommand (Highest Priority)

When the user drags a shape, every `MouseMove` produces a delta. The command stores only the total displacement, not every intermediate step. The `CanMergeWith` method ensures consecutive moves on the same shape collapse into one undo step.

File: MapCanvas/Core/Commands/MoveShapeCommand.cs

```
public class MoveShapeCommand : ICommand
{
    private readonly ShapeManager _shapeManager;
    private readonly IShape _shape;
    private PointD _totalDelta; // Mutable: grows during merge

    public string Description =>
        $"Move {_shape.GetType().Name}";

    public MoveShapeCommand(ShapeManager shapeManager,
        IShape shape, PointD delta)
    {
        _shapeManager = shapeManager;
        _shape = shape;
        _totalDelta = delta;
    }

    public void Execute()
    {
        _shape.Translate(_totalDelta);
        _shapeManager.RebuildSpatialIndex();
    }

    public void Undo()
    {
        var reverse = new PointD(-_totalDelta.X, -_totalDelta.Y);
        _shape.Translate(reverse);
        _shapeManager.RebuildSpatialIndex();
    }

    public void Redo() => Execute();

    public bool CanMergeWith(ICommand other)
    {
        // Merge consecutive moves on the SAME shape
    }
}
```

```

        return other is MoveShapeCommand move
            && move._shape.Id == _shape.Id;
    }

    public void MergeWith(ICommand other)
    {
        if (other is MoveShapeCommand move)
        {
            _totalDelta = new PointD(
                _totalDelta.X + move._totalDelta.X,
                _totalDelta.Y + move._totalDelta.Y);
        }
    }
}

```

IShape Needs a Translate Method

Your IShape interface currently has no Translate method. You need to add: `void Translate(PointD delta);` to IShape, then implement it in the abstract Shape base class by offsetting all vertices. See Section 4.6 for the exact change.

4.2 MoveMultipleShapesCommand

When the user selects multiple shapes and drags them together:

```

public class MoveMultipleShapesCommand : ICommand
{
    private readonly ShapeManager _shapeManager;
    private readonly List<IShape> _shapes;
    private PointD _totalDelta;

    public string Description =>
        $"Move {_shapes.Count} shapes";

    public MoveMultipleShapesCommand(ShapeManager shapeManager,
        List<IShape> shapes, PointD delta)
    {
        _shapeManager = shapeManager;
        _shapes = new List<IShape>(shapes);
        _totalDelta = delta;
    }

    public void Execute()
    {
        foreach (var s in _shapes) s.Translate(_totalDelta);
        _shapeManager.RebuildSpatialIndex();
    }

    public void Undo()

```

```

    {
        var reverse = new PointD(-_totalDelta.X, -_totalDelta.Y);
        foreach (var s in _shapes) s.Translate(reverse);
        _shapeManager.RebuildSpatialIndex();
    }

    public void Redo() => Execute();
    public bool CanMergeWith(ICommand other)
        => other is MoveMultipleShapesCommand;
    public void MergeWith(ICommand other)
    {
        if (other is MoveMultipleShapesCommand m)
            _totalDelta = new PointD(
                _totalDelta.X + m._totalDelta.X,
                _totalDelta.Y + m._totalDelta.Y);
    }
}

```

4.3 ModifyPropertyCommand (Generic)

This is a generic command that can undo ANY property change on any shape: color, layer name, fill, visibility, or any custom property in the Properties dictionary.

```

public class ModifyPropertyCommand<T> : ICommand
{
    private readonly IShape _shape;
    private readonly string _propertyName;
    private readonly T _oldValue;
    private readonly T _newValue;
    private readonly Action<IShape, T> _setter;

    public string Description =>
        $"Change {_propertyName}";

    public ModifyPropertyCommand(IShape shape,
        string propertyName, T oldValue, T newValue,
        Action<IShape, T> setter)
    {
        _shape = shape;
        _propertyName = propertyName;
        _oldValue = oldValue;
        _newValue = newValue;
        _setter = setter;
    }

    public void Execute() => _setter(_shape, _newValue);
    public void Undo() => _setter(_shape, _oldValue);
    public void Redo() => Execute();
    public bool CanMergeWith(ICommand other) => false;
}

```

```
    public void MergeWith(ICommand other) { }  
}
```

Usage Examples:

```
// Change border color  
var cmd = new ModifyPropertyCommand<Color>(  
    shape, "BorderColor",  
    shape.BorderColor, Color.Red,  
    (s, v) => s.BorderColor = v);  
_undoManager.ExecuteCommand(cmd);  
  
// Change layer name  
var cmd = new ModifyPropertyCommand<string>(  
    shape, "LayerName",  
    shape.LayerName, "Roads",  
    (s, v) => s.LayerName = v);  
_undoManager.ExecuteCommand(cmd);
```

Why Generic?

Instead of creating ChangeColorCommand, ChangeLayerCommand, ChangeFillCommand, ChangeVisibilityCommand as separate classes, one generic class handles all property changes. Less code, same undo capability.

4.4 CompositeCommand (Transaction/Group)

When a single user action involves multiple sub-operations that must undo together. Example: splitting a parcel requires deleting the original and adding two new parcels. Pressing Ctrl+Z must reverse ALL THREE operations in one step.

```
public class CompositeCommand : ICommand
{
    private readonly List<ICommand> _commands;
    private readonly string _description;

    public string Description => _description;

    public CompositeCommand(string description,
        params ICommand[] commands)
    {
        _description = description;
        _commands = new List<ICommand>(commands);
    }

    public void Execute()
    {
        foreach (var cmd in _commands)
            cmd.Execute();
    }

    public void Undo()
    {
        // Undo in REVERSE order (critical!)
        for (int i = _commands.Count - 1; i >= 0; i--)
            _commands[i].Undo();
    }

    public void Redo() => Execute();
    public bool CanMergeWith(ICommand other) => false;
    public void MergeWith(ICommand other) { }
}
```

Usage Example: Parcel Split

```
// Future parcel split operation:
var splitCmd = new CompositeCommand("Split Parcel",
    new DeleteShapesCommand(shapeManager,
        new List<IShape> { originalParcel }),
    new AddShapeCommand(shapeManager, leftParcel),
    new AddShapeCommand(shapeManager, rightParcel)
);
_undoManager.ExecuteCommand(splitCmd);
// Ctrl+Z removes both new parcels and restores original
```

4.5 ImportShapesCommand

Your DXF import already uses BulkAddShapesCommand which is correct. But a dedicated ImportShapesCommand can carry additional metadata (source file name, timestamp) for better undo descriptions:

```
public class ImportShapesCommand : ICommand
{
    private readonly ShapeManager _shapeManager;
    private readonly List<IShape> _shapes;
    private readonly string _sourceFile;

    public string Description =>
        $"Import {_shapes.Count} shapes from {Path.GetFileName(_sourceFile)}";

    public ImportShapesCommand(ShapeManager shapeManager,
        IEnumerable<IShape> shapes, string sourceFile)
    {
        _shapeManager = shapeManager;
        _shapes = shapes.ToList();
        _sourceFile = sourceFile;
    }

    public void Execute() => _shapeManager.BulkAddShapes(_shapes);
    public void Undo() => _shapeManager.BulkRemoveShapes(_shapes);
    public void Redo() => Execute();
    public bool CanMergeWith(ICommand other) => false;
    public void MergeWith(ICommand other) { }
}
```

4.6 Required Change: Add Translate to IShape

The MoveShapeCommand requires shapes to know how to move themselves. Add this to your IShape interface and implement in the Shape base class:

Add to IShape.cs:

```
/// <summary>
/// Move the shape by the given delta in world coordinates.
/// </summary>
void Translate(PointD delta);
```

Implement in each shape class. Example for LineShape:

```
public void Translate(PointD delta)
{
```



```
        Start = new PointD(Start.X + delta.X, Start.Y + delta.Y);  
        End = new PointD(End.X + delta.X, End.Y + delta.Y);  
    }
```

Example for PolylineShape:

```
public void Translate(PointD delta)  
{  
    for (int i = 0; i < _vertices.Count; i++)  
    {  
        _vertices[i] = new PointD(  
            _vertices[i].X + delta.X,  
            _vertices[i].Y + delta.Y);  
    }  
}
```

Chapter 5: Improvements to UndoRedoManager

5.1 Fix the Max Undo Level Enforcement

Your current code for enforcing max undo levels converts the stack to an array, clears it, and re-pushes everything minus the oldest. This is $O(n)$ and reverses the stack temporarily. Here is a cleaner approach using a bounded collection:

Current Bug

Your current implementation does: `var commands = _undoStack.ToArray(); _undoStack.Clear();` then re-pushes in reverse. This works but is fragile. If `_maxUndoLevels` is small (say 10) and the user works fast, you are rebuilding the entire stack on every operation after the 10th.

Fix: Use LinkedList Instead of Stack

```
// Replace Stack<ICommand> with LinkedList<ICommand>
private readonly LinkedList<ICommand> _undoStack = new();
private readonly LinkedList<ICommand> _redoStack = new();

public void ExecuteCommand(ICommand command)
{
    command.Execute();

    // Try merge with previous
    if (_undoStack.Count > 0
        && _undoStack.Last.Value.CanMergeWith(command))
    {
        _undoStack.Last.Value.MergeWith(command);
        return;
    }

    _undoStack.AddLast(command);
    _redoStack.Clear();

    // Trim oldest - O(1) operation
    while (_undoStack.Count > _maxUndoLevels)
        _undoStack.RemoveFirst();
}

public void Undo()
{
    if (_undoStack.Count == 0) return;
    var cmd = _undoStack.Last.Value;
    _undoStack.RemoveLast();
    cmd.Undo();
    _redoStack.AddLast(cmd);
}

public void Redo()
```

```
{
    if (_redoStack.Count == 0) return;
    var cmd = _redoStack.Last.Value;
    _redoStack.RemoveLast();
    cmd.Redo();
    _undoStack.AddLast(cmd);
}
```

Why LinkedList?

Stack only allows push/pop from one end. To trim the oldest command, you need access to the bottom. LinkedList gives O(1) AddLast, RemoveLast (undo/redo), AND RemoveFirst (trim oldest). No array rebuilds.

5.2 Add Events to UndoRedoManager

Your toolbar has Undo and Redo buttons, but they do not enable/disable based on stack state. Add events so the UI can react:

```
// Add to UndoRedoManager:
public event EventHandler? StateChanged;

// Fire after every ExecuteCommand, Undo, Redo, Clear:
StateChanged?.Invoke(this, EventArgs.Empty);
```

Wire in DrawingCanvasControl:

```
_undoManager.StateChanged += (s, e) =>
{
    toolStripButton6.Enabled = _undoManager.CanUndo; // Undo btn
    toolStripButton7.Enabled = _undoManager.CanRedo; // Redo btn
    toolStripButton6.ToolTipText =
        $"Undo {_undoManager.GetUndoDescription()}";
    toolStripButton7.ToolTipText =
        $"Redo {_undoManager.GetRedoDescription()}";
};
```

Chapter 6: Wiring It All Together

6.1 Keyboard Shortcuts

Add these to your existing `ProcessCmdKey` method in `DrawingCanvasControl`:

```
// In ProcessCmdKey, add these cases:

// Ctrl+Z = Undo (shape operations)
if (keyData == (Keys.Control | Keys.Z))
{
    _undoManager.Undo();
    _deferredRenderer.RenderNow();
    panelCanvas.Invalidate();
    return true;
}

// Ctrl+Y = Redo
if (keyData == (Keys.Control | Keys.Y))
{
    _undoManager.Redo();
    _deferredRenderer.RenderNow();
    panelCanvas.Invalidate();
    return true;
}

// Alt+Left = Previous View
if (keyData == (Keys.Alt | Keys.Left))
{
    var prev = _viewHistory.GoBack();
    if (prev.HasValue)
    {
        _engine.RestoreViewState(prev.Value);
        _deferredRenderer.RenderNow();
        panelCanvas.Invalidate();
    }
    return true;
}

// Alt+Right = Next View
if (keyData == (Keys.Alt | Keys.Right))
{
    var next = _viewHistory.GoForward();
    if (next.HasValue)
    {
        _engine.RestoreViewState(next.Value);
        _deferredRenderer.RenderNow();
        panelCanvas.Invalidate();
    }
    return true;
}
```

```
}  
  
// Escape = Cancel active drawing (already partial)  
if (keyData == Keys.Escape)  
{  
    CancelActiveDrawing();  
    return true;  
}
```

Chapter 7: Complete File Map

File	Status	Action
Core/ViewState.cs	NEW	Create (readonly struct)
Core/ViewHistoryManager.cs	NEW	Create (back/forward navigation)
Core/Commands/MoveShapeCommand.cs	NEW	Create (with merge support)
Core/Commands/MoveMultipleShapesCommand.cs	NEW	Create (group move)
Core/Commands/ModifyPropertyCommand.cs	NEW	Create (generic property change)
Core/Commands/CompositeCommand.cs	NEW	Create (transaction grouping)
Core/Commands/ImportShapesCommand.cs	NEW	Create (enriched import)
Core/Commands/UndoRedoManager.cs	MODIFY	LinkedList + StateChanged event
Core/Commands/ICommand.cs	UNCHANGED	Already complete
Core/Commands/ShapeCommands.cs	UNCHANGED	Keep all 4 existing commands
Core/DrawingEngine.cs	MODIFY	Add GetViewState() + RestoreViewState()
Models/Shapes/IShape.cs	MODIFY	Add Translate(PointD delta) method
Models/Shapes/Shape.cs	MODIFY	Implement Translate in base class
Models/Shapes/LineShape.cs	MODIFY	Override Translate
Models/Shapes/PolylineShape.cs	MODIFY	Override Translate
Models/Shapes/RectangleShape.cs	MODIFY	Override Translate
Models/Shapes/CircleShape.cs	MODIFY	Override Translate
Models/Shapes/EllipseShape.cs	MODIFY	Override Translate
Models/Shapes/TextShape.cs	MODIFY	Override Translate
DrawingCanvasControl.cs	MODIFY	Add _viewHistory, keyboard shortcuts, events

Chapter 8: Implementation Order

Do these in exact order. Each step is testable independently.

Step 1: Fix UndoRedoManager (30 minutes)

1. Replace Stack with LinkedList in UndoRedoManager
2. Add StateChanged event
3. Fire event after ExecuteCommand, Undo, Redo, Clear
4. Wire event in DrawingCanvasControl to enable/disable toolbar buttons
5. Test: draw 3 shapes, undo 3 times, redo 3 times. Buttons should toggle correctly.

Step 2: View History (45 minutes)

6. Create ViewState.cs
7. Create ViewHistoryManager.cs
8. Add GetViewState() and RestoreViewState() to DrawingEngine
9. Add _viewHistory field to DrawingCanvasControl
10. Add RecordView calls after zoom wheel, pan end, and zoom extents
11. Add Alt+Left and Alt+Right keyboard shortcuts
12. Test: zoom in 5 times, Alt+Left 3 times (should go back 3 zoom levels), Alt+Right 2 times.

Step 3: Translate Method (45 minutes)

13. Add void Translate(PointD delta) to IShape interface
14. Implement Translate in LineShape, PolylineShape, RectangleShape, CircleShape, EllipseShape, TextShape
15. Test: manually call shape.Translate(new PointD(100, 100)) on each shape type, verify bounding box shifts correctly

Step 4: Move Commands (1 hour)

16. Create MoveShapeCommand.cs
17. Create MoveMultipleShapesCommand.cs
18. These are ready for use once you implement the drag tool (selection + mouse drag)
19. For now, test via code: create a MoveShapeCommand, ExecuteCommand, Undo, verify position returns

Step 5: Property and Composite Commands (30 minutes)

20. Create `ModifyPropertyCommand.cs` (generic)
21. Create `CompositeCommand.cs`
22. Test `ModifyPropertyCommand`: change a shape color, undo, verify original color restores
23. Test `CompositeCommand`: create a composite with 2 `AddShapeCommands`, undo both in one step

Step 6: Import Command (15 minutes)

24. Create `ImportShapesCommand.cs`
25. Replace `BulkAddShapesCommand` usage in `btnImportDxf_Click` with `ImportShapesCommand`
26. Test: import DXF, undo should show description like "Import 247 shapes from parcels.dxf"

Chapter 9: Future Commands (Post Version 1.0)

These commands are not needed for Version 1.0 but the architecture you build now supports them without any changes to UndoRedoManager or ICommand:

Future Command	Uses CompositeCommand?	Needs New Class?
SplitParcelCommand	Yes (delete 1 + add 2)	No, use CompositeCommand
MergeParcelCommand	Yes (delete 2 + add 1)	No, use CompositeCommand
RotateShapeCommand	No	Yes (stores angle delta)
ScaleShapeCommand	No	Yes (stores scale factor)
ChangeLayerBatchCommand	No	Yes (or use CompositeCommand with ModifyProperty)
ResizeShapeCommand	No	Yes (stores old/new vertex positions)
ReorderLayerCommand	No	Yes (stores old/new draw order)
ContributionCalculateCommand	Possibly	Yes (stores before/after areas)

Architecture Flexibility

Notice that CompositeCommand eliminates the need for most special-purpose commands. Parcel split, merge, and batch operations are all composites of existing Add/Delete/Modify commands. You only need new command classes for operations with unique undo logic (like rotation, which stores an angle, not a list of sub-commands).

Summary

What	Effort	Impact
Fix UndoRedoManager (LinkedList + events)	30 min	Enables button state, cleaner trim
ViewHistoryManager (zoom/pan undo)	45 min	Alt+Left/Right view navigation
Translate method on all shapes	45 min	Prerequisite for move commands
MoveShapeCommand + MoveMultiple	1 hour	Drag-to-move with full undo
ModifyPropertyCommand (generic)	15 min	Undo any property change
CompositeCommand	15 min	Groups operations for atomic undo
ImportShapesCommand	15 min	Better undo descriptions for imports

Total estimated effort: approximately 4 hours of focused work.

After implementation, your RePlot canvas will support: Ctrl+Z and Ctrl+Y for all shape operations (add, delete, move, property change, import, clear), Alt+Left and Alt+Right for view navigation (zoom/pan history), automatic toolbar button enable/disable, command merging for smooth drag operations, composite commands ready for parcel split and merge operations, and a clean architecture where every future operation simply creates a command object.

Your existing ICommand interface and command pattern are correctly designed. This guide does not replace them. It completes them by adding the missing command classes and the view history system that separates document changes from navigation changes.